

University of Brighton

RO501 Coursework - Jake Watson

Name: Jake Oliver Watson
Student ID: 24808713
Course: Robotics and Mechatronics BEng
Module: RO501 - Intelligent Systems
Assignment: RO501 Coursework
Submission Date: 23rd January 2026
Word Count: < 2000 (excl Figures, tables, abstract)

Abstract

This report contains research into Intelligent Agents, including the components of such agents, developing an Intelligent Agent, and the ethics of Intelligent Agents with both research and deployment in mind. Each part of this paper is treated as an independent report, the references the one previous. Part A of the report is the components of an agent, and covers topics like PEAS to an introduction to neural networks. Part B covers designing an intelligent agent, this section specifically looks at deploying an agent to drive a race car. Finally Part C reflects on part B by looking at the ethical implications of such an agent.

Contents

1 Part A - Understanding Intelligent Agents 2
1.1 Components of a Intelligent Agent 2
1.2 Comparison of Agent Types 3
1.3 Explanation of how ANNs can enhance Intelligent Agents 4
1.4 Real World Uses 5
1.5 Conclusion 6
2 Part B - Developing an Intelligent Agent 6
2.1 Agent Architecture 6
2.2 Data Analysis and Training 7
2.3 Results 8
3 Conclusion 9
4 Part C - Ethics of Intelligent Agents 9
4.1 Ethics of UBAC 9
4.2 Conclusion 10
5 Final Conclusion 11
6 References 11
7 Appendix 13

1 Part A - Understanding Intelligent Agents

The definition of an Intelligent Agent is "An advanced agent that can learn, adapt, and improve its actions based on past experiences and reasoning"[1]. With this in mind, it is possible to then continue and explore the different types of intelligent agents. Mathematically an intelligent agent f can be defined as a function that maps perception P^* to action A.

$$f : P^* \rightarrow A \tag{1}$$

1.1 Components of a Intelligent Agent

A good way to describe the components of an Intelligent Agent when referring to AI is to use the acronym PEAS[2]. PEAS stands for the following:

- **P** Performance Measure: A measurement of success and performance, this can be used to help improve the agent.
- **E** Environment: The area in which the agent can perceive and act.
- **A** Actuators: The part of the agent that allows interaction in the environment.
- **S** Sensors: The receptive inputs to the agent that allows it perceive its environment.
- **RL** Reasoning Logic: This is essentially the "brain" of agent, it decides how to act through planning and reasoning.

PEAS can be used to infer the components required for an Intelligent Agent. Although in the above definition an extra term was used **RL**, this term was added because PEAS is more of a framework to build agents, and as such, it tends to abstract away from the reasoning and planning steps which are crucial in making intelligent agents, intelligent.

1.2 Comparison of Agent Types

Under the Russel and Norvig[3] classification of intelligent agents, 5 types exist, each type has a different level complexity from the one that precedes it. These are: Simple Reflex Agents (SRA), Model Based Reflex Agents (MBRA), Goal Based Reflex Agents (GBRA), Utility Based Reflex Agents (UBRA), and finally Learning Agents (LA).

SRA An agent whose action is based on the current perception, actions are picked using fixed conditions. As a result of these fixed conditions SRAs are very computationally efficient, unfortunately these conditions it makes SRA only suited to specific tasks where environments are predictable, and fully observable. Use case can include; Thermostat

MBRA An agent which features an internal model of how the world changes, and can track how its actions may affect the world around it, using this model, it can track how unobservable parts of the environment may change over time. These agents use the current observation and the model to make decisions. Comparing to SRAs, MBRAs are significantly more robust and well adapt at tasks. However, they require many more resources. Use case can include; autonomous car,

GBRA Instead of picking actions purely based off of the current observation and internal model, GBRA will specifically pick actions that push them closer to achieving their goals, this allows the model to plan and put foresight into its decisions. Similarly to MBRAs, GBRAs have an internal model that can track how the world changes. Once again, computational costs go up. Use case of these can include; Warehouse Robotics.

UBRA Utility based agents build on top of the goal approach, by instead taking in to account how desirable different outcomes maybe. This allows for more intelligent decision making, and allows for planning that not only achieves their goal but can surpass it. This allows the agent to be more robust in undesirable situations. Once again, this increases computational costs. Use cases can include; Delivery Drones.

LA Learning based agents don't start off with any predefined rules but instead learn their own through interactions with the environment, slowly improving over time. Learning Agents are much more computationally expensive, and can take prolonged periods of time to train but can often excel in tasks. Use case can include; Home Robot

To summarise, as agent types get more sophisticated, robustness and reliability can greatly improve, often outperforming humans. However this comes at a cost of more powerful computing hardware and longer building times.

1.3 Explanation of how ANNs can enhance Intelligent Agents

As Artificial Neural Networks (ANNs) are Universal Function Approximators (UFA)[4] they can estimate any function that is given to them, in fact ANNs do not need to know the true function like Taylor or Fourier series do, instead all they require is a sample of the function output and inputs. ANNs are mathematically defined as a non linear mapping function f that maps an input $\mathbb{R}^n$ to an output $\mathbb{R}^m$.

$$f : \mathbb{R}^n \rightarrow \mathbb{R}^m \tag{2}$$

Referring to both equation 1 and 2 shows a striking a resemblance, that given an input both an intelligent agent and an ANN will attempt to map it to a corresponding output. Using this fact, it can be suggested that an ANN can be used to replace or enhance the internal mechanisms of a intelligent

agent that may not already use them. Further more, as ANNs are UFA it further pushes the idea that ANNs could outperform hard coded algorithms by learning to adapt to both, unexpected events or events that were never considered, this can be observed when ANNs learn to reward hack in order to adapt.[5]

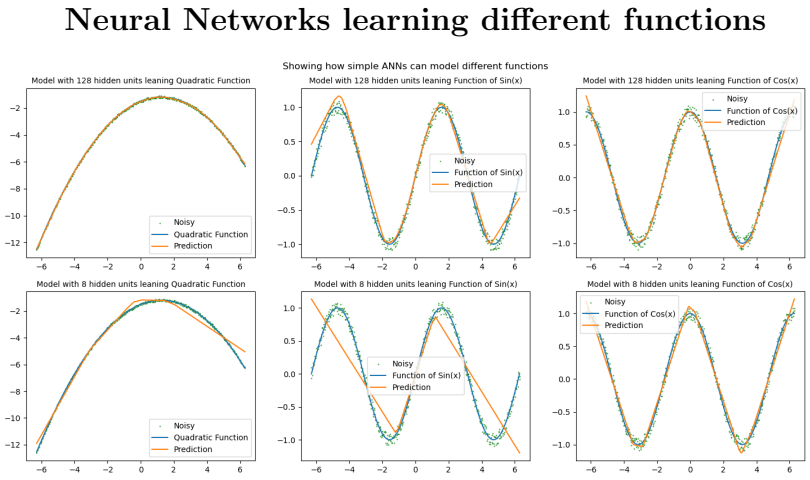


Figure 1: ANNs trained on noisy data points (Green), and showing how the parameters in the hidden layer can improve their ability to approximate any function - Credit Jake Oliver Watson - GitHub.

1.4 Real World Uses

Real world applications of Intelligent Agents would be autonomous vehicles. One of the first of these was ALVINN[6], an agent designed to assess the use of ANNs in autonomous vehicles. Nearly 30 years after ALVINN a team at Nvidia[7] created an agent named DAVE2. Both ALVINN and DAVE2, under modern terminology, would be classed as SRA's. DAVE2 even suggested that SRA's with ANN enhancement could achieve a reliable autonomy in real world situations, these models have even paved the way for more modern approaches in end-to-end learning approaches for self driving cars.

1.5 Conclusion

To conclude, Intelligent Agents require 5 key components to be classed as an Intelligent Agent, these components are best described through the PEAS+RL acronym, an acronym that includes the reasoning logic that many agents use. Comparisons were also made between the different types of agents, comparing the internal mechanisms and the scenarios in which they may be deployed. Further more, a discussion occurred on the use cases of neural networks being used to enhance intelligent agents. In conclusion this section has demonstrated a good understanding of LO1.

2 Part B - Developing an Intelligent Agent

This section shall demonstrate LO2 and LO4, following the design and implementation of an intelligent agent. Everything from architecture design to testing the robustness of the agent shall be measured, and discussed. For this section, the intention is to replicate the results in the aforementioned papers[6,7] by using the Udacity Car Simulator[8]. For sake of clarity of we shall refer to this agent as UBAC (University of Brighton Autonomous Car).

2.1 Agent Architecture

UBAC requires two functions to operate, these being; A speed controller, and a steering controller. For the speed controller, a simple PID controller was used and then tuned to stability. This gave reliable and user controllable speeds. For the steering controller, a neural network was used, figure 3 shows this architecture which primarily consists of many 'blocks' stacked together, although it does omit the initial dimension scaling head to increase channel count. Each block features two sets of convolutions with padding to ensure the dimensionality remains unchanged, layer normalisation is then used as this was found to improve model performance. In total this model consists of $\approx 480k$ parameters making it comparable to DAVE2, in terms of model size. Applying modern agent classification shows that UBAC is an SRA, this is due to its fixed conditional ruling and lack of any complex internal modelling which are found in more complex models.

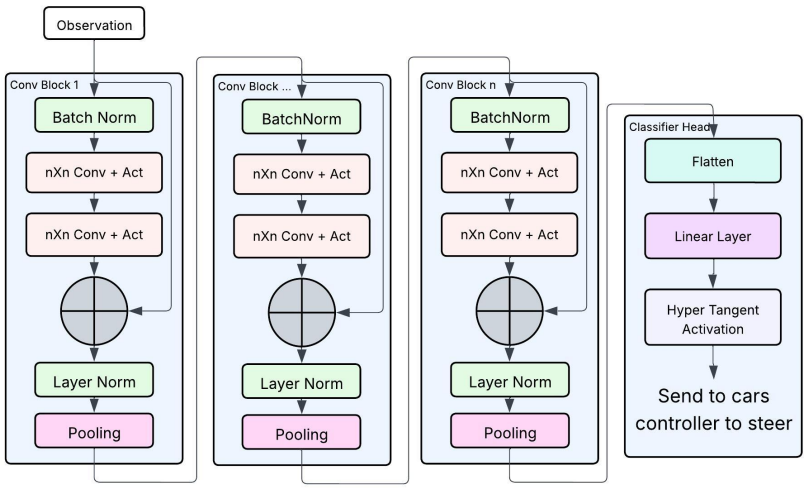


Figure 2: UBAC consists of stacked convolutional blocks with residual connections, layer normalisation and pooling before feeding these features into regressive or classification head.

2.2 Data Analysis and Training

The training data for UBAC was collected through the interface provided in the Udacity self driving car simulator. To enhance dataset variance, we sampled every third frame of the $150k$ initial frames. This reduction in sampling frequency ensures that the model learns from distinct state-action transitions rather than highly correlated consecutive frames. To supplement this, and make a more suitable distribution, frequency clipping was performed to reduce bias and increase variance in the dataset. Further more, cropping was done to remove extra information that maybe visible in the frame, such as; environment skyline, and the vehicle hood. The PID was tuned through a process of trial and error, slowly adjusting values until optimal stability with low steady state error is occurred. Please refer to the appendix for, training parameters, dataset steering angle plot, and loss curve.

Example of dataset balancing.

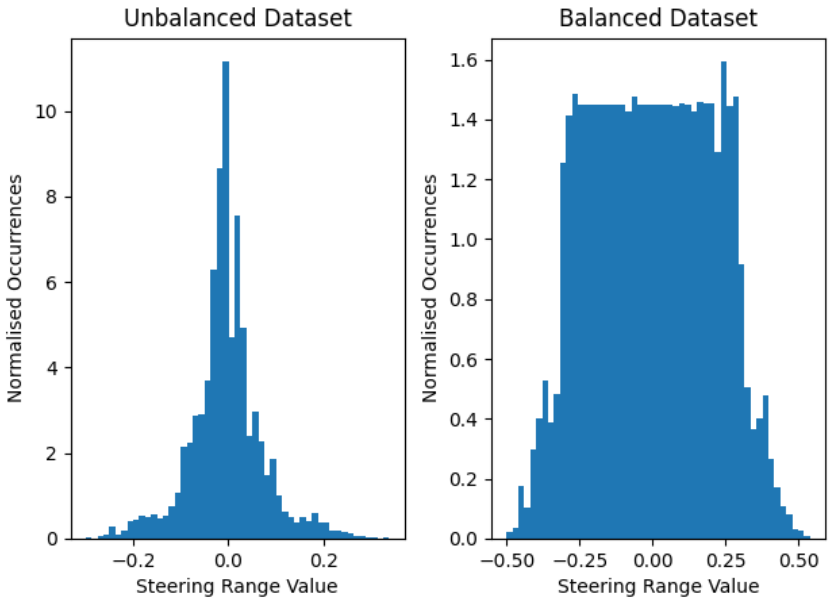


Figure 3: Dataset was balanced using stochastic methods based on the frequency of occurrences

Figure 3 shows the dataset after balancing, notice how the unbalanced dataset is severely over represented for 0 degree driving angles. Statistical balancing was performed, this allowed for more variance and guaranteed that over represented values would be evenly present through out the dataset.

2.3 Results

To rate the performance of UBAC two metrics were used, the autonomy equation from the DAVE2 paper and the reliability formula both given with;

$$\text{autonamy}[\%] = \left(1 - \frac{(\text{numer of interventions}) \cdot 6}{\text{elapsed time [seconds]}}\right) \cdot 100 \tag{3}$$

$$\text{reliability}[\%] = \left(1 - \frac{\text{interventions}}{\text{total actions} + \text{interventions}}\right) \cdot 100 \tag{4}$$

Thus giving us two performance metrics, one which can be compared to the performance of DAVE2 and one which shows how reliable the agent is. Unfortunately due to stability reasons, tests were unable to be conducted on other test tracks, therefore it is crucial to note that UBAC may perform differently on different test tracks. In conclusion though, UBAC got an average autonomy and reliability of 85% and 96.7%.

3 Conclusion

While UBAC demonstrated strong results, it is important to remember that UBAC was both trained and tested on one single track, therefore further testing must be done in order to further confirm its performance. To finally conclude, both LO2 and LO4 objectives have been met in this section. Discussing architecture choices, dataset balancing and refinement, and finally training and testing.

4 Part C - Ethics of Intelligent Agents

Ethics of an intelligent agent is one of the largest limitations that prevents deployments of such systems. Ethics doesn't just cause moral implications, it also causes legal ones that need to be addressed. This section will attempt to tackle ethics and legality of Intelligent Agents by using UBAC as an example.

4.1 Ethics of UBAC

The ethics of an AV (Autonomous Vehicle) is a complex ethical and legal minefield filled with many challenges. A good starting point however would be classify the system that such an agent would be, UBAC would be an L2 system[9]. Such a system may not act without a driver present, and the driver must be paying attention. With this definition in mind, it is possible to list out the issues that can plague such systems, and suggest options that maybe able to mitigate such risks.

Ethical & Legal Challenges	Description	Mitigations
Trolley Problem	Given a situation with undesirable outcomes, which one should the agent pick.	No mitigations it is immoral to place weightings on a life, however, sacrifice the vehicle as driver consents to risk of system and vehicle have safety features.
Overeliance	Driver becoming over reliant on these system.	Driver awareness software built in to detect loss of attention, steering wheel detection, or distraction detection. Giving control back should also be a seamless procedure.
Road Legal Compliance	Potential requirements maybe required before deployment on vehicles.	Strict investigations into legal compliance, and the structure that governs regulations on L2 systems, to ensure UBAC is compliant to such things.
GDPR & Data protection	GDPR requires strict policies to prevent data from being misused, add hearing to this policy is essential.	No data collection users data stays local, furthermore a policy of no accounts and open sourcing of software pushes responsibility off of UBAC.
Bias & Fairness	Ensuring the model is unbias and fair.	A model witch is bias or unfair will cause safety issues, ensuring the dataset shows a fair and even distribution of circumstances will make the model more reliable and fair. Testing should take place after production to probe for bias.
Liability & Accountablility	Who is liable for accidents, and who should be held accountable.	UBAC is marketed as a driver assistance tool and requires full user attention, therefore blame will fall on to the operator of the vehicle.

Table 1: Shows the challenges that will be faced, a description of them, and a way to mitigate the risks

4.2 Conclusion

While mitigations to the proposed challenges have been labelled it is still important to discuss and acknowledge that these issues do exists. Whether its a legal challenge or one that poses a hazard to life or property, understanding the issues created and assessing if the risk is worth it is always an important step. Therefore this report concludes that an intelligent agent should only be adopted if it improves a system, that is to say greatly improves the safety. With this in mind and by using available reports[10], it is with this intent in mind that systems like UBAC should be available for adoption assuming they improve road safety and have systems dedicated to ensuring driver awareness at all times.

5 Final Conclusion

This paper demonstrates a comprehensive understanding of intelligent agents across three parts. Part A explores the theoretical components of intelligent agents and how ANNs enhance their capabilities, supported by graphical and mathematical examples. Part B applies this theory by designing and implementing an intelligent agent for race car control, documenting the development process, errors encountered, and mitigation strategies. Part C examines the ethical and legal implications, concluding that deployment is justified only when intelligent agents demonstrably improve upon existing systems. Therefore it is safe to assume that all learning objectives have been covered fully, and to a satisfactory level.

6 References

1. Ali R. RO401 - Intelligent Agents. 2024;

2. GeeksforGeeks. Understanding PEAS in Artificial Intelligence [Internet]. GeeksforGeeks. 2020. Available from: https://www.geeksforgeeks.org/artificial-intelligence/understanding-peas-in-artificial-intelligence/

3. IBM. Types of AI Agents [Internet]. Ibm.com. 2025. Available from: https://www.ibm.com/think/topics/ai-agent-types

4. Hornik K, Stinchcombe M, White H. Multilayer feedforward networks are universal approximators. Neural Networks [Internet]. 1989 Jan;2(5):359–66. Available from: https://researchers.dellmed.utexas.edu/en/publications/multilayer-feedforward-networks-are-universal-approximators

5. Baker B, Kanitscheider I, Markov T, Wu Y, Powell G, McGrew B, et al. Emergent Tool Use From Multi-Agent Autocurricula. arXiv:190907528 [cs, stat] [Internet]. 2020 Feb 10; Available from: https://arxiv.org/abs/1909.07528

6. Pomerleau D. ALVINN: AN AUTONOMOUS LAND VEHICLE IN A NEURAL NETWORK [Internet]. 1988. Available from: https://proceedings.neurips.cc/paper/1988/file/812b4ba287f5ee0bc9d43bbf5bbe87fb-Paper.

pdf

7. Bojarski M, Testa D, Dworakowski D, Firner B, Flepp B, Goyal P, et al. End to End Learning for Self-Driving Cars [Internet]. arXiv.org. 2016. Available from: https://arxiv.org/abs/1604.07316

8. udacity/self-driving-car-sim [Internet]. GitHub. 2021. Available from: https://github.com/udacity/self-driving-car-sim

9. Staff SE. The 6 Levels of Vehicle Autonomy Explained [Internet]. Synopsys.com. 2025. Available from: https://www.synopsys.com/blogs/chip-design/autonomous-driving-levels.html#3

10. Cicchino JB. Convenience or safety system? Crash rates of vehicles equipped with partial driving automation. Traffic Injury Prevention. 2025 Feb 21;1–11.

7 Appendix

Table 2: Model Training Hyperparameters

Category	**Parameter**	**Value**
Training Setup	Epochs	100
	Batch Size	32
Optimization	Optimizer	AdamW
	Weight Decay	1×10^{-3}
LR Strategy	Learning Rate (Initial)	1×10^{-3}
	LR Scheduler	Cosine Annealing
	Minimum LR	1×10^{-5}

Loss graph of model used in demonstrations.

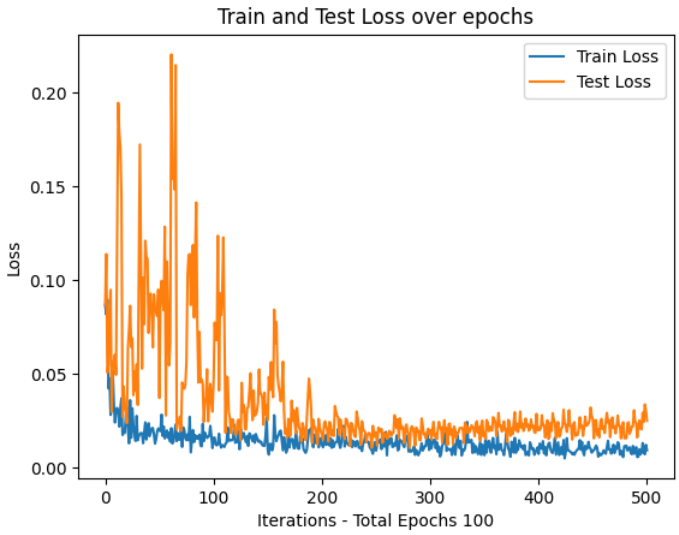


Figure 4: This is the loss plot of the CNN used in UBAC, observe the unstable test loss due to poor hyper-parameter values. Eventually settles down due to LR scheduler.

Steering angles with respect to frames passed.

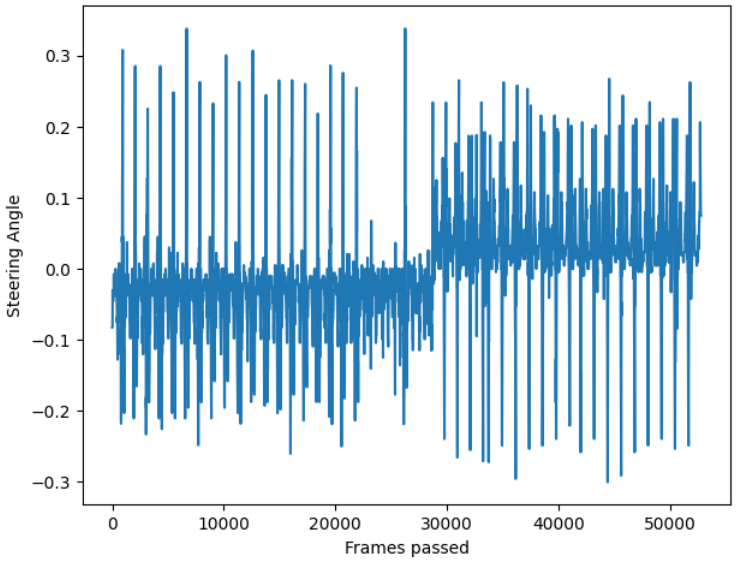


Figure 5: This is the steering angle plot with respect to passed frames.

Image preprocessing.

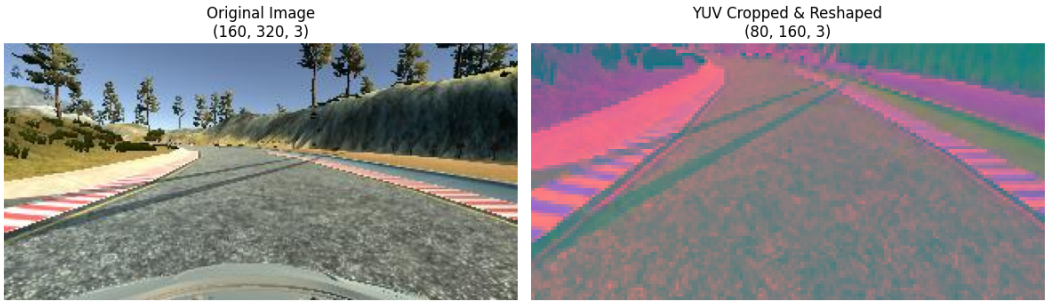


Figure 6: This here shows the processing done to image, the left images in classic RGB format while the right is in YUV and has had redundant information removed.